A PROJECT REPORT ON

DIABETES RISK PREDICTION USING MACHINE LEARNING

with Patient Grouping and an Interactive Dashboard

Submitted in partial fulfilment of the requirements for the course

Machine Learning

Submitted by

[Saurabh Pandey]

[12604659]

Under the guidance of

[Mr Pradeep Kumar Sir]

[M.Tech Data Science]

[Lovely Professional University]

[2026-2028]

Declaration

I confirm that this report was written by me and is about my machine learning project. All code and work done to create this report was completed by me. Data set is the property of its owner. Papers and software can be found in the reference section.
[Saurabh Pandey] Date: [12/11/2026]

Abstract

Diabetes is a chronic disease that people tend to not know they have until its progressed. This project aims to create a model that can predict if someone may have diabetes based on some given attributes. The dataset used was the Pima Indians Diabetes Dataset which contains 768 patients and 8 attributes for each patient including blood sugar level, BMI, age and number of pregnancies they have had.

The process used to complete this task follows typical steps used when working with machine learning. We began by preprocessing our data there were many 0s in some of the columns that would be an impossible value for that attribute so we assumed those were null values and imputed the columns with their median. After preprocessing we did some exploratory data analysis using graphs and correlation we capped off any outliers and created 3 new features. We then built 6 different classifiers Logistic Regression, KNN, Decision Tree, Random Forest, SVM and Naive Bayes. All of these were fit using 5 fold cross validation. Once we had our models built we tested them on 154 unseen patients.

The highest scoring models achieved a ROC-AUC score of around .82-.83 and were able to identify around 3/4 of patients that were diabetic. We did some hyper parameter tuning with gridsearchcv on the random forest and used feature importance to determine the top attributes. We also clustered our data using kmeans to get 3 clusters. There was one cluster that was significantly less likely to be diabetic than the other clusters. Lastly I built a streamlit web app to display my findings and allow a user to test a new patient.

Keywords: machine learning, diabetes prediction, classification, data cleaning, cross-validation, clustering, Streamlit

Contents

Declaration 2
Abstract 3
1. Introduction 5
1.1 Background 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope 5
2. Dataset Description 6
2.1 Source 6
2.2 Features 6
2.3 Data Quality Observations 6
3. Tools and Technologies 7
4. Methodology and Workflow 8
4.1 Data Cleaning 8
4.2 Feature Engineering and Data Split 8
4.3 Evaluation Metrics 9
4.4 Project Structure 9
5. Exploratory Data Analysis 10
5.1 Class Balance 10
5.2 Distribution of Features 10
5.3 Correlation Analysis 11
5.4 Outlier Detection 12
6. Model Development 13
6.1 Models Selected 13
6.2 Training and Cross-Validation 13
6.3 Hyperparameter Tuning 13
6.4 Clustering and PCA 13
7. Results 14
7.1 Confusion Matrix 14
7.2 Tuned Random Forest 14
7.3 Important Features 15
7.4 Patient Groups 15
8. Model Comparison 17
8.1 Cross-Validation Comparison 17
8.2 Test Set Comparison 17
8.3 ROC Curves 18
8.4 Which Model to Prefer 18
9. Dashboard 19
10. Limitations and Ethical Considerations 20
10.1 Limitations 20
10.2 Ethical Considerations 20
11. Conclusion and Future Scope 21
11.1 Conclusion 21
11.2 Future Scope 21
12. GitHub and LinkedIn Evidence 22
References 23

1. Introduction

1.1 Background

Diabetes occurs when your body is unable to regulate its blood sugar levels. If you don't receive proper treatment it can lead to heart failure kidney failure blindness and nerve damage. Many people with diabetes don't even know they have the disease. Screenings are a good way to find out if someone has diabetes but can be costly and take time as well as require skilled workers.

With machine learning we can approach this differently. We can use information about patients with known outcomes to predict if a patient may have diabetes based on simple readings. It will not be able to diagnose someone with diabetes but could allow medical workers to identify those at higher risk.

1.2 Problem Statement

Predict if someone has diabetes given the values of there glucose levels blood pressure, insulin, bmi and age. We would also want to know what are the important values that effect whether they have diabetes or not. This should be able to explain itself to a non machine learning person.

1.3 Objectives

- Loading and cleaning a real life medical dataset and getting it ready to be used.
- Viewing the data through graphs and analysis.
- Fitting multiple models for classification and validating them through cross validation and testing on a hold out dataset.
- Evaluating our models by accuracy, precision, recall, f1 score and ROC-AUC. Explaining why we should look at recall for this problem.
- Tuning the model and looking at what features play a role in the model.
- Clustering patients with out the knowledge of if they have diabetes or not using K-means clustering and PCA.
- Create a simple dashboard to display our findings and save our model.

1.4 Scope

There will only be one dataset used in this project and we will be classifying if a person is diabetic or not. This is simply a project to learn how to do a full machine learning project from start to finish. This should not be used to diagnose any one.

2. Dataset Description

2.1 Source

This is the Pima Indians Diabetes Dataset from Kaggle. You can find this dataset here [9] and also from the UCI repository. This data was gathered by the National Institute of Diabetes and Digestive and Kidney Diseases [1]. Each patient in this dataset is a female at least 21 years old of Pima Indian heritage. There are 768 total instances in this dataset. Of those 768 there are 268 (34.9%) that have diabetes and 500 (65.1%) that do not.

2.2 Features

Table 1: Columns in the dataset

Column	Meaning	Type
Pregnancies	Number of times the patient was pregnant	Number
Glucose	Plasma glucose concentration after a glucose tolerance test	Number
BloodPressure	Diastolic blood pressure (mm Hg)	Number
SkinThickness	Triceps skin fold thickness (mm)	Number
Insulin	2-hour serum insulin (mu U/ml)	Number
BMI	Body mass index (weight in kg / height in m squared)	Number
DiabetesPedigree	A score showing family history of diabetes	Number
Age	Age in years	Number
Outcome	Target: 1 = diabetic, 0 = not diabetic	Label

2.3 Data Quality Observations

No missing data in the table. The problem with the data set is that there is a 0 value in five of the columns to represent no value. If someone's blood pressure or BMI was 0 they would be dead. Table 2 lists the amount of zeros in the data set.

Table 2: Zeros that actually represent missing values

Column	Number of zeros	Share of 768 rows
Glucose	5	0.7%
BloodPressure	35	4.6%
SkinThickness	227	29.6%
Insulin	374	48.7%
BMI	11	1.4%

Since insulin and skin thickness had the most missing values I decided to not delete the rows because that would take up close to half of my data. That is why I imputed the missing values.

3. Tools and Technologies

The whole project is written in Python. The main libraries and their purpose are given in Table 3.

Table 3: Libraries used

Library	**Used for**
NumPy [3]	Numerical operations on arrays
Pandas [4]	Loading the CSV file, cleaning and handling tables
Matplotlib [5] and Seaborn [6]	Drawing graphs
Scikit-learn [2]	Scaling, splitting, models, metrics, cross-validation, GridSearchCV, PCA and K-Means
Joblib [7]	Saving and loading trained models
Streamlit [8]	Building the web dashboard
Visual Studio Code	Writing and running the code

4. Methodology and Workflow

The project is divided into small steps, and each step is written in its own Python file. The output of one file is saved and then read by the next one. This keeps the work easy to follow, because anyone can see exactly what happened to the data at each stage. Figure 1 shows the complete workflow.

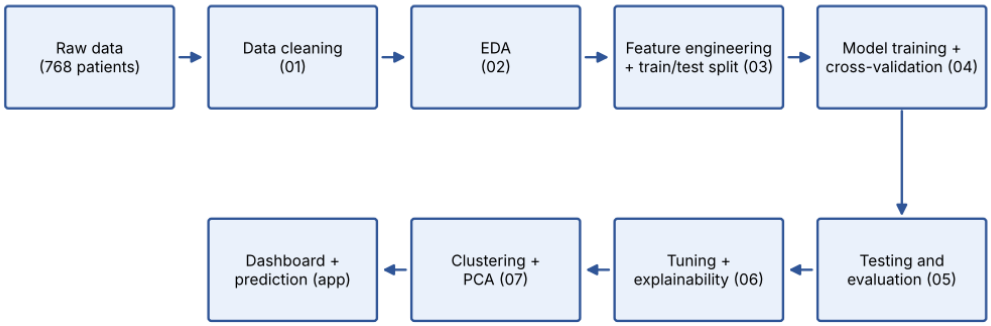


Figure 1: Workflow of the project (numbers in brackets are the file numbers)

4.1 Data Cleaning

The raw file has no header row, so the column names were added while loading. The zeros in Glucose, BloodPressure, SkinThickness, Insulin and BMI were replaced with missing values. The missing values in each column received their replacement through the calculation of median values from their respective columns. The researchers selected median values instead of average values because Insulin and SkinThickness measurements showed extreme values which affected the average but kept the median values steady. The data analysis showed that there were no duplicate entries which appeared in the dataset. The cleaned data was saved as a new file so that the raw file always stays untouched.

4.2 Feature Engineering and Data Split

Extreme values were capped at the IQR limits (see Section 5.4), and three new features were added to give the models more useful information:

- **AgeGroup:** age divided into four bands (up to 30, 31 to 45, 46 to 60, above 60).
- **BMICategory:** BMI divided into underweight, normal, overweight and obese, using the usual cut-off values of 18.5, 25 and 30.
- **Glucose_BMI:** glucose multiplied by BMI and divided by 100, to capture patients who have both values high.

After this, the data had 11 input features. It was split into a training set of 614 patients (80%) and a test set of 154 patients (20%). A stratified split was used, which means that both parts

have the same share of diabetic patients: 34.9% in training and 35.1% in testing. The test set was kept aside and was used only for the final evaluation.

Feature scaling was not applied at this stage on purpose. It was placed inside the training pipeline instead, so that the scaler learns its mean and spread only from the training data. If scaling is done on the whole dataset first, some information from the test set leaks into training and the final scores look better than they really are.

4.3 Evaluation Metrics

The models were tested on the unseen test set using the following measures.

- **Accuracy:** the share of all patients that were classified correctly.
- **Precision:** out of the patients predicted as diabetic, how many really were.
- **Recall:** out of the patients who really are diabetic, how many the model found.
- **F1-score:** a single number that balances precision and recall.
- **ROC-AUC:** how well the model separates the two classes across all possible thresholds. A value of 0.5 is random guessing and 1.0 is perfect.

In a health problem recall deserves special attention. A false alarm only leads to one extra test, but a missed patient may go without treatment. For this reason a model with a slightly lower accuracy and a higher recall can be the better choice.

4.4 Project Structure

All files are kept in one folder, and every file does one job. The files are meant to be run in order, and a single script (run_all.py) can run all of them together. Keeping cleaning, training and testing in separate files means that a mistake can be traced to a specific step quickly, and a step can be repeated without running everything again.

Table 4: Files of the project

File	Purpose
config.py	File paths, column names, random seed and other common settings
01_data_cleaning.py	Loads the raw data, fixes missing values, removes duplicates
02_eda.py	Creates the graphs and prints statistics and outlier counts
03_feature_engineering.py	Caps outliers, adds new features, splits into train and test files
04_train_models.py	Trains the six models with cross-validation and saves them
05_test_evaluate.py	Tests the models on the test file and draws ROC curves and the confusion matrix
06_tuning_explainability.py	GridSearchCV, feature importance and the final saved model
07_clustering_pca.py	K-Means clustering, silhouette score and PCA plot
predict.py	Predicts the risk for one patient from the command line

File	Purpose
app.py	Streamlit dashboard
run_all.py	Runs all the steps in order

5. Exploratory Data Analysis

Before building models, the cleaned data was studied using a class count, histograms, a correlation heatmap and boxplots. The aim was to understand the shape of the data and to see which measurements seem related to diabetes.

5.1 Class Balance

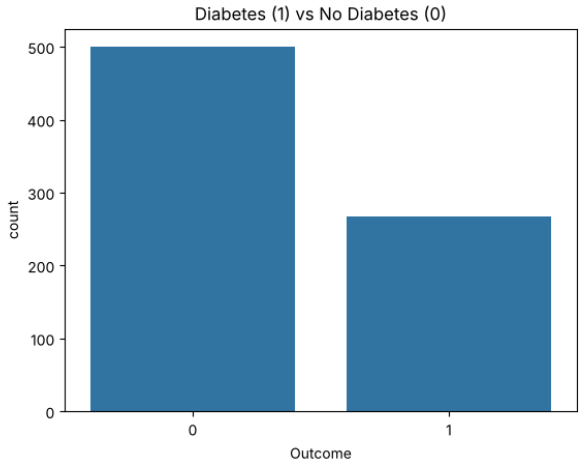


Figure 2: Number of non-diabetic (0) and diabetic (1) patients

Figure 2 shows that the classes are not equal: there are almost twice as many non-diabetic patients as diabetic ones. If this is ignored, a model can reach a fair accuracy just by guessing "not diabetic" for everybody, which would be useless in practice. For this reason balanced class weights were used in the models.

5.2 Distribution of Features

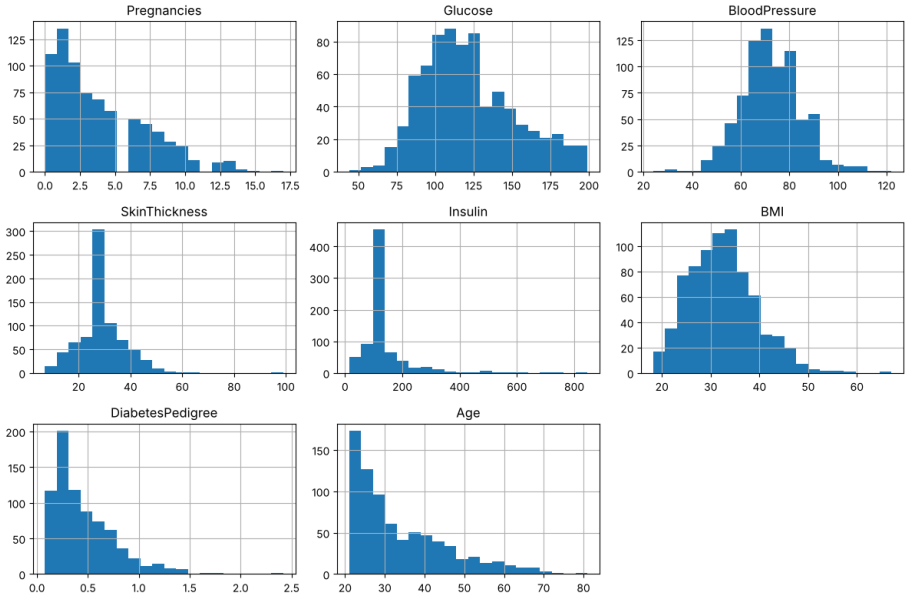


Figure 3: Distribution of each feature

The histograms show that Pregnancies, Age, Insulin and DiabetesPedigree are skewed to the right, meaning most patients have small values and a few have very large ones. Glucose and BloodPressure look closer to a bell shape.

5.3 Correlation Analysis

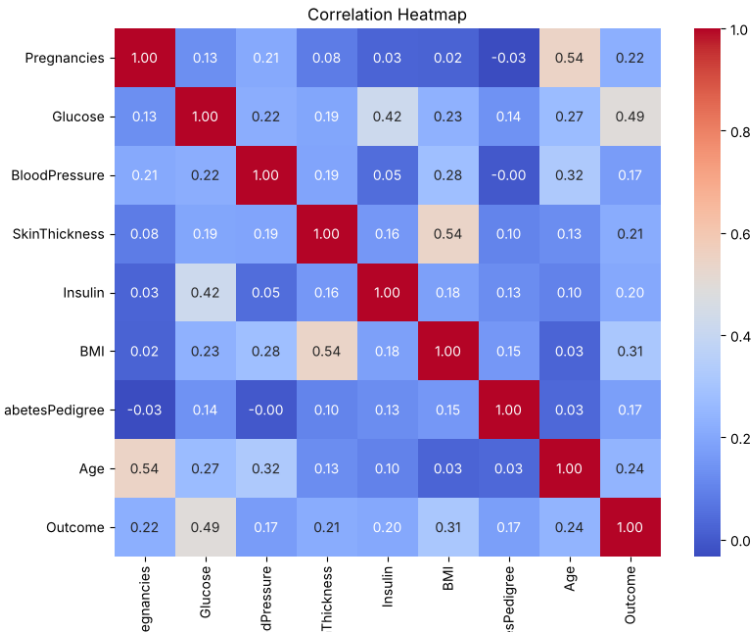


Figure 4: Correlation heatmap

The correlation of each feature with the Outcome column is given in Table 5. Glucose has the strongest link with diabetes, followed by BMI and Age. No single feature has a very high correlation with the result, which tells us that a combination of features will be needed for good predictions.

Table 5: Correlation of each feature with Outcome

Feature	Correlation
Glucose	0.493
BMI	0.312
Age	0.238
Pregnancies	0.222
SkinThickness	0.215
Insulin	0.204
DiabetesPedigree	0.174
BloodPressure	0.166

5.4 Outlier Detection

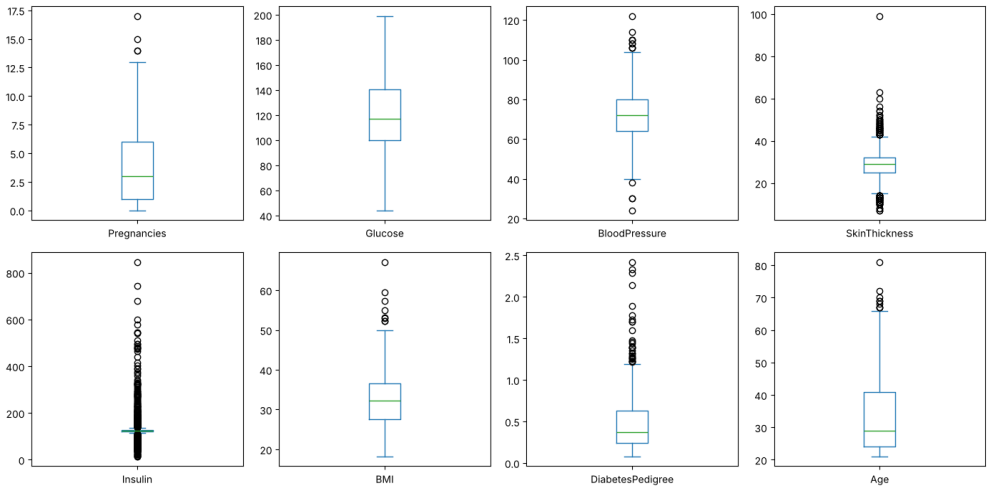


Figure 5: Boxplots used to look for outliers

Outliers were counted with the interquartile range (IQR) rule, in which a value is treated as extreme if it lies more than 1.5 times the IQR outside the middle 50% of the data. Insulin had the most extreme values (346), followed by SkinThickness (87) and DiabetesPedigree (29). Since the dataset is small, the extreme rows were not deleted. They were capped at the IQR limits instead, so that no patient was lost.

6. Model Development

6.1 Models Selected

Six classification algorithms were trained. They were chosen because they work in different ways, which makes the comparison more interesting.

Table 6: Models used

Model	Basic idea
Logistic Regression	Calculates the probability of diabetes from a weighted sum of the features
K-Nearest Neighbours	Looks at the 7 most similar patients and takes a vote
Decision Tree	Asks a series of yes/no questions (depth limited to 4 to avoid over-fitting)
Random Forest	Builds 200 different trees and combines their answers
Support Vector Machine	Finds the boundary that separates the two classes with the widest gap
Naive Bayes	Uses probability and assumes that the features are independent

6.2 Training and Cross-Validation

The pipeline consisted of models which StandardScaler processed before training data evaluation through stratified 5-fold cross-validation. The method splits training data into five sections which the model uses for training on four sections while testing on the remaining section until all sections serve as test data. The average ROC-AUC values from five rounds served as the basis for comparing different models. The algorithm operates with equal class weight distribution which it supports to handle classification problems. All trained pipelines were saved with Joblib.

6.3 Hyperparameter Tuning

The GridSearchCV process delivered an optimized setup for the Random Forest model through its search operation. The search process evaluated two different tree counts (100 and 200) alongside three maximum depth options (3, 5, 8) and three minimum sample requirements for leaf nodes (1, 5, 10). The research team created 18 unique parameter sets which they tested by running five cross-validation cycles to assess their performance with ROC-AUC as the evaluation metric.

6.4 Clustering and PCA

The research team applied K-Means clustering to standardised data which they cleaned before removing the Outcome column to check if patients would naturally form distinct clusters. The researchers evaluated cluster numbers through the elbow method and silhouette score to select three clusters for their analysis. The data underwent PCA reduction into two dimensions which

enabled the researchers to visualize their clustered information through a graphical representation. The researchers calculated group average values after clustering to identify the specific patient characteristics which their groups represented.

7. Results

7.1 Confusion Matrix

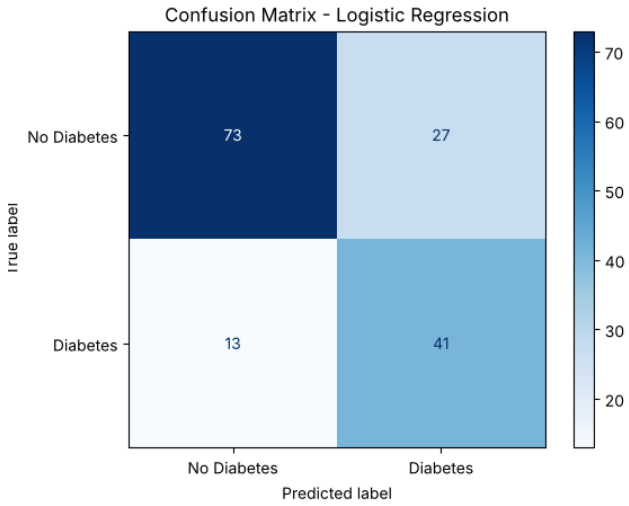


Figure 6: Confusion matrix of Logistic Regression (best model in cross-validation)

The Logistic Regression model produced a test set confusion matrix which identified 73 patients without diabetes correctly while it falsely identified 27 non-diabetic patients as diabetic and accurately detected 41 diabetic patients but failed to detect 13 diabetic patients. The model identified 41 out of 54 cases which results in a 76% recall rate and it produced 68 positive results which included 41 actual cases for a 60% precision rate. The system successfully identifies most diabetic patients but it produces false positive alerts which occur four times for every ten actual warnings.

7.2 Tuned Random Forest

GridSearchCV selected a Random Forest with 100 trees, a maximum depth of 5 and at least 10 samples in each leaf. The model achieved a cross-validation ROC-AUC score of 0.846 which demonstrated better performance than the original forest model which scored 0.817. The default forest creates deep trees which capture noise from the limited training data but the restricted tree depth helps the model develop better generalization abilities.

Table 7: Tuned Random Forest on the test set

Measure	Value
ROC-AUC	0.826
Accuracy	0.73
Precision (diabetic)	0.59
Recall (diabetic)	0.76
F1-score (diabetic)	0.67

The test set shows that the tuned forest with a score of 0.826 performs similarly to the untuned version which scored 0.829. The training data showed improvement through tuning yet the test data results failed to surpass previous results. The training provides essential knowledge about how parameter optimization produces minimal changes when working with small data sets yet validation results tend to differ from test results because of random factors.

7.3 Important Features

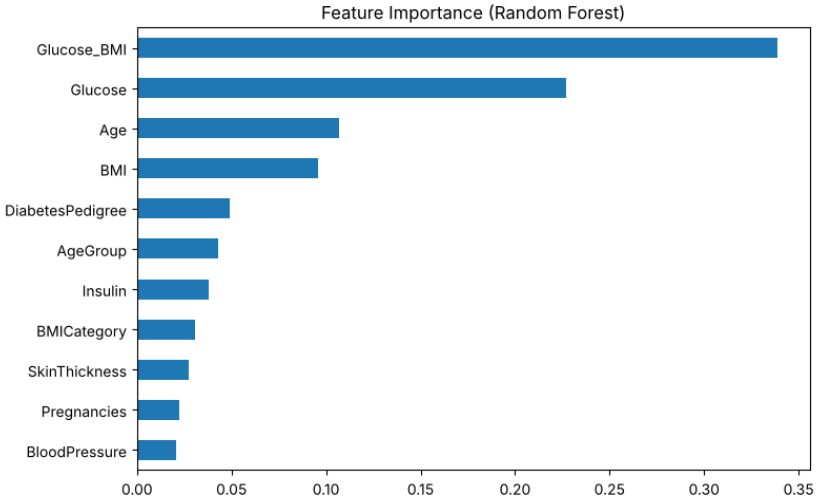


Figure 7: Feature importance in the tuned Random Forest

The forest established Glucose_BMI (0.339) and Glucose (0.227) as its top features which followed by Age (0.107), BMI (0.096) and DiabetesPedigree (0.049). The study revealed that glucose levels together with BMI measurements and age data appeared as the most important factors according to the correlation analysis. The Glucose_BMI feature which I created became the most powerful single input variable which proved that feature creation techniques produce valuable results. The Random Forest model assigns higher importance scores to features which have multiple distinct values so I performed a second evaluation through permutation importance on the test data. The procedure requires you to shuffle each column independently while tracking the score reduction during this process. The method discovered the same vital features which included Glucose_BMI and Age and BMI but it presented them through a different sequence thus confirming the main finding.

7.4 Patient Groups

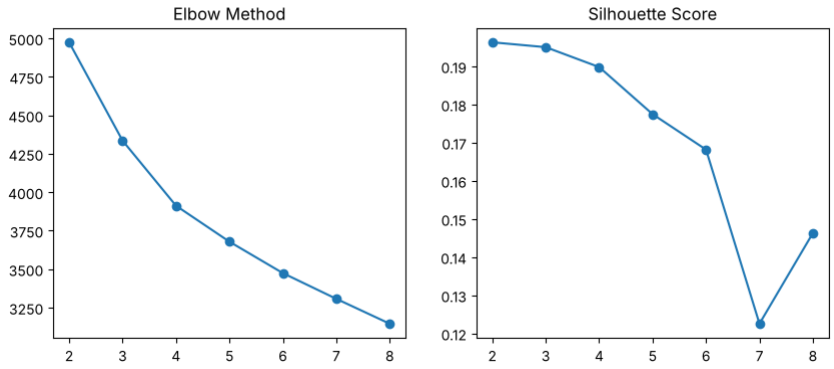


Figure 8: Elbow curve and silhouette score for different numbers of clusters

The silhouette analysis revealed that every cluster count selection showed weak performance because the method generated scores which remained near 0.20 for 2 and 3 clusters while decreasing for higher cluster numbers. Medical data typically shows overlapping groups because patients undergo slow transitions between different medical conditions which results in low cluster evaluation scores. Three clusters were chosen because they gave an easy-to-explain division and a score about the same as two clusters.

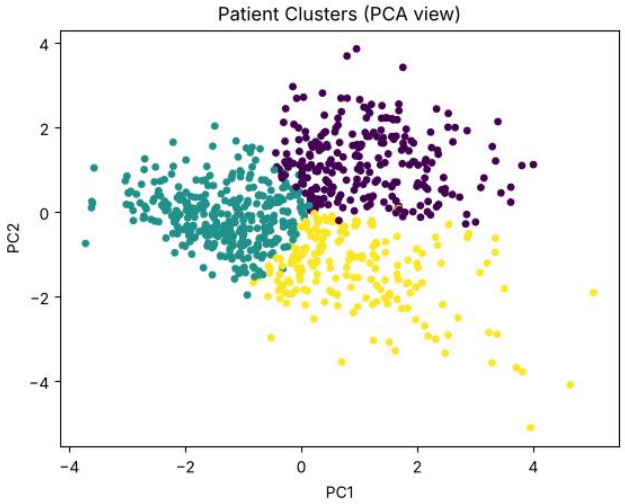


Figure 9: The three patient clusters shown on two PCA axes

Table 8: Average values in each cluster

Cluster	Patients	Avg age	Avg pregnancies	Avg glucose	Avg BMI	Avg insulin	Diabetes rate
0	248	46.0	7.4	130.8	32.8	139.6	52%
1	335	25.9	2.3	106.6	28.5	113.3	13%
2	185	29.3	1.9	136.6	39.2	191.6	52%

The research team decided not to include the Outcome column during their clustering process but the diabetes rates between different groups showed substantial variation. The first cluster

contains patients who are younger and they show reduced glucose levels together with lower BMI values yet only 13% of these patients develop diabetes. The older group consists of patients who experienced multiple pregnancies while the younger group shows extremely high BMI values at 39.2 together with elevated insulin levels. The older group consists of patients who experienced multiple pregnancies while the younger group shows extremely high BMI values at 39.2 together with elevated insulin levels. The older patients with multiple pregnancies belong to cluster 0 while cluster 2 consists of younger patients who show extremely high BMI values at 39.2 and elevated insulin levels. The older patients with multiple pregnancies belong to cluster 0 while cluster 2 consists of younger patients who show extremely high BMI values at 39.2 and elevated insulin levels. The older patients with multiple pregnancies belong to cluster 0 while cluster 2 consists of younger patients who show extremely high

8. Model Comparison

8.1 Cross-Validation Comparison

Table 9 shows the average ROC-AUC of each model on the training data during cross-validation. A smaller spread between the folds means the model is more stable.

Table 9: 5-fold cross-validation ROC-AUC (training data)

Model	Mean ROC-AUC	Spread (+/-)
Logistic Regression	0.850	0.010
Naive Bayes	0.837	0.018
Support Vector Machine	0.836	0.024
Random Forest (untuned)	0.817	0.032
K-Nearest Neighbours	0.807	0.030
Decision Tree	0.786	0.040

Logistic Regression had the highest score and also the smallest spread, so it was the best model during cross-validation. The simple Decision Tree was the weakest, which is expected because one shallow tree cannot capture many patterns.

8.2 Test Set Comparison

Table 10: Performance on the 154 unseen test patients

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Support Vector Machine	0.766	0.641	0.759	0.695	0.831
Random Forest (untuned)	0.734	0.607	0.685	0.643	0.829
K-Nearest Neighbours	0.779	0.717	0.611	0.660	0.824
Logistic Regression	0.740	0.603	0.759	0.672	0.818
Naive Bayes	0.734	0.603	0.704	0.650	0.804
Decision Tree	0.662	0.512	0.759	0.612	0.774

Several points can be seen from Table 10. The ranking on the test set is a little different from the ranking in cross-validation: the Support Vector Machine has the best ROC-AUC (0.831) and Logistic Regression is fourth (0.818). The top four models show only small variations between them. The test group included 154 patients but the position of two or three patients in the results could change the entire ranking so none of these models proves to be better than the others. K-Nearest Neighbours has the highest accuracy (0.779) but it finds only 61% of the diabetic patients. The Support Vector Machine, Logistic Regression and Decision Tree all reach a recall of 0.759. The Decision Tree achieves high recall rates but it produces the most false alarms because its precision score (0.512) remains at the lowest level.

8.3 ROC Curves

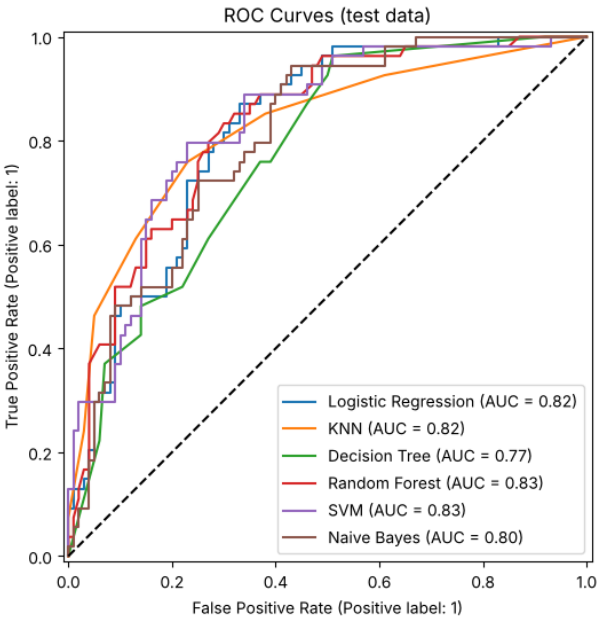


Figure 10: ROC curves of all models on the test data

The ROC curves in Figure 10 lie well above the diagonal line, which stands for random guessing, so every model has learned useful patterns. The curves of the top models are close to each other, which agrees with Table 10.

8.4 Which Model to Prefer

Because the cost of missing a diabetic patient is high, the Support Vector Machine and Logistic Regression give the best balance in this project: both find about 76% of diabetic patients with a fair precision. The cross-validation results show Logistic Regression as the most stable model while maintaining its simple explanation method through direct feature weight interpretation. The Random Forest becomes the best selection when researchers need to determine which features hold the most importance. The selection of leading models requires researchers to focus on explanation abilities and recall performance instead of minor ROC-AUC differences during their analysis of this limited dataset.

9. Dashboard

A dashboard was built with Streamlit [8] so that the results can be seen without opening the code. It runs in the web browser and has five tabs.

- **Overview:** number of patients, share of diabetic patients, the best model, the cleaned table and summary statistics.
- **EDA:** class balance, correlation heatmap, boxplots and histograms.
- **Model Results:** the metric table of all models, a bar chart, the ROC curves, the confusion matrix and feature importance.
- **Patient Groups:** the PCA plot of clusters and the elbow and silhouette graphs.
- **Predict:** a form in which the values of one patient are entered, and the saved model returns the probability of diabetes along with a low-risk or high-risk message.

For example, when a 45-year-old patient with glucose 150, BMI 33.0, blood pressure 80, insulin 120, skin thickness 30, 2 pregnancies and a pedigree value of 0.6 was entered, the model gave a diabetes risk of about 72% and showed a high-risk message. The new features (age group, BMI category and Glucose_BMI) are calculated by the app itself, so the user needs to type only the eight original measurements. The trained model is stored with Joblib, so the dashboard does not need to train the model again each time it is opened.

10. Limitations and Ethical Considerations

10.1 Limitations

- The dataset is small (768 patients). Small test sets give results that move up and down easily, as seen in the difference between the cross-validation and test rankings.
- All patients are women of Pima heritage aged 21 or above. A model trained on this group may not work properly for men, for younger people or for other populations.
- A large part of the Insulin and SkinThickness values were missing and were filled with the median. This is a simple method and it may hide real differences between patients.
- The silhouette scores of the clusters are low, so the groups are a convenient summary and not sharply separate types of patients.
- The models use only eight measurements. Real diagnosis depends on many other things such as lifestyle, diet, medical history and repeated tests.

10.2 Ethical Considerations

Health information stands as highly confidential data which requires strict privacy protection. The actual system needs patients to give permission for their records to receive protected storage while staff must remove all personal identifiers from these documents. The dataset which I use in this study contains public information that does not reveal any personal identification data. Fairness is another concern. The data comes from one population, so the model may be less accurate for others, and a wrong or missing prediction can have a serious effect on a person. The tool should therefore be used only as a support for a doctor and never as the final decision. The dashboard displays a message which states that the system output does not represent an official medical diagnosis. Lastly, the model can be explained. The tool shows its most important elements which doctors and patients can use to understand why the system produced a high-risk assessment thus building trust through meaningful explanations.

11. Conclusion and Future Scope

11.1 Conclusion

The project developed an entire machine learning system which predicts diabetes risk through its complete operational process. The first stage of data processing involved cleaning the original information before conducting exploration and creating additional features which scientists used to separate the data into training and testing segments. The research team trained six different models which they evaluated through cross-validation before testing their performance with new patient data. The superior models achieved ROC-AUC scores between 0.82 and 0.83 while detecting 76% of diabetic patients but they generated false alarms during 40% of their warning signals. All testing methods demonstrated that glucose levels and body mass index together with patient age stood out as the most critical elements. Clustering analysis revealed three distinct patient groups which showed different diabetes prevalence rates and Streamlit dashboard provided an easy-to-use system for displaying results and prediction information. The research study uncovered several practical insights which showed that zeros function as hidden missing data points and classification accuracy becomes unreliable for datasets with imbalanced classes and scaling operations should occur inside the pipeline to prevent data leakage and small datasets show that different successful models produce results which fall into regular statistical fluctuations.

11.2 Future Scope

- Test the models on a larger and more varied dataset, including male patients and other age groups.
- Try boosting methods such as XGBoost or LightGBM and compare them with the models used here.
- Use SHAP values to explain the prediction for each individual patient.
- Try better ways of filling missing values, such as KNN imputation, and check whether the scores improve.
- Choose the decision threshold on purpose, for example lowering it to raise recall, and study the effect on false alarms.
- Add hierarchical clustering and DBSCAN to compare with K-Means.
- Put the dashboard online so that it can be used from any device.

References

[1] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," Proceedings of the Symposium on Computer Applications in Medical Care, 1988, pp. 261-265.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[3] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[4] W. McKinney, "Data structures for statistical computing in Python," Proceedings of the 9th Python in Science Conference, pp. 56-61, 2010.

[5] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[6] M. L. Waskom, "seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

[7] Joblib Development Team, "Joblib: running Python functions as pipeline jobs," https://joblib.readthedocs.io.

[8] Streamlit documentation, https://docs.streamlit.io.

[9] Pima Indians Diabetes Database, National Institute of Diabetes and Digestive and Kidney Diseases, distributed on Kaggle and the UCI Machine Learning Repository.

Github-Link = https://github.com/Saurabh8227/diabetes-risk-prediction

Linkedin-Link = https://www.linkedin.com/feed/update/urn:li:activity:7526200408168390656/